

7.15 release candidate, assembly, merge cadence and upstream preparation plan

Date: 2026-09-29. Plan only: nothing is merged, nothing is sent upstream, no device was flashed, and develop is untouched. It follows the retro (715-master-retro-20260929.md) and its main finding, that six accepted block heads are not in the chain that Blocks 8 to 13 were built on.

1. What can be tested on a device today

Candidate ref: candidate/7.15.0-lineage-20260929, a branch on the fork pointing at 603fb904f (Block 13). It contains Blocks 4, 5 and 7 to 13. It does **not** contain the audited repairs from 00a, 00b, 01, 02, 03 and 06 (retro section 4), so it is a valid test of what it contains and not the release.

Artifacts (hosted run 36622461353, exact head 603fb904f; every one of the 23 files per variant matches its arm-build-manifest.json):

Variant	firmware.keepkey.bin	Size	SHA-256
Full	firmware.keepkey.v7.15.0-603fb90-full-firmware.keepkey.bin	623,860	906a3703f2f0ffef7c22a2297724bf85c8c4ab8ab2fb25674823d2386475ee51
Bitcoin-only	firmware.keepkey.v7.15.0-603fb90-bitcoin-only-firmware.keepkey.bin	363,628	b35ab3c18168ec2154744598790a01631c12c6096d562f65a4b3c6abf2aa8a16

Both builds are unsigned (the header's signature slots are empty). Consequences, separating what was verified from what is inferred:

- Verified in the code: tools/firmware/keepkey_main.c calls flash_collectHWEntropy(SIG_OK == sigRet). An unsigned build therefore takes the non-privileged branch, uses the fixed entropy bytes and never touches the OTP block. **The Block 11e first-boot OTP path cannot be exercised by any unsigned build.** It needs a signed build from the release ceremony.

- Inferred, not tested: that entropy feeds the PIN key derivation, so a device provisioned by signed firmware may fail to unlock or be wiped after an unsigned flash, which matches the ledger's note that unsigned candidates cannot prove storage preservation. Use a test device and a throwaway seed. I did not flash anything and will not without you.

Worth checking by hand (expected results come from the tests, not from a device): recovery cipher entry, including an empty character and a delete; the press-free entropy request and its budget; an Osmosis send with a long IBC denomination (expect several confirm pages, all showing the exact amount); Hive account and transfer consent pages, including memo paging and a custom chain ID; refusal of wallet creation on future-format storage; and auto-lock behaviour during a signing flow. OLED readability and power-cycle behaviour are release gates that no block has claimed.

2. Assembly plan

Merging each missing head into the Block 13 tip (`git merge -tree`, nothing written; merge bases all dated 2026-09-21):

Head	PR	Conflicted files	Notes
00a 9cb72384f	#844	13	includes keepkey_flash.c, keepkey_main.c, reset.c, both workflows
00b 8126f802f	#856	72	30 in lib/firmware, 12 in include/keepkey, 12 in unittests/firmware
01 752466b25	#831	8	ethereum.c, fsm.c, home_sm.c and tests
02 9237d0a30	#855	11	messages.c, ethereum.c, fsm.c, recovery and USB tests, the Python pin
03 5aa047e4d	#833	13	adds storage.c and storage.cpp
06 9a177aa21	#859	0	merges clean

Options. (1) Merge the six heads into the tip in ascending order, one audited unit each, resolving toward the later block where it re-audited the same code and keeping the earlier block's repair everywhere else. (2) Rebuild the chain from `deve lop` and replay Blocks 4 to 13, which touches more code and discards the heads already audited. (3) Declare the early repairs superseded, which the presence data does not support without a per-finding proof.

Recommendation: option 1, as units A-00a, A-00b, A-01, A-02, A-03 and A-06 (06 first, since it is clean). Each unit follows the same gate the last three blocks used: merge, resolve, preflight, hosted CI on both variants, artifact verification, an independent read-only audit of the resolution, Copilot under the severity rule, and a failing control for each conflict area. Its size is comparable to Block 11, and 00b (72 files) is the largest single step.

Done means: every accepted head is an ancestor of the release tip; the presence check reports at least 99% for each block, with every remaining missing line justified per file as superseded; hosted CI is green at the tip with artifacts verified; and the device kit is rebuilt from that tip.

3. Merge cadence into `deve lop` and the release branch

Practiced so far (dry runs only): the ordered forecast above; and the per-unit gate itself, run for thirteen units across Blocks 11 to 13. **Not practiced, deliberately:** any merge into fork `deve lop`, which the SOP forbids under rehearsal authorization. Fork and upstream `deve lop` are both `fc1e93746` today.

Sequence after assembly. (1) Cut `release/7.15.0-rc28` from the assembled tip (`rc27` is the last existing candidate). (2) Hosted CI and artifact verification at that head, then a signed build for device testing. (3) One cumulative release-product PR from the release branch to `deve lop`, carrying a manifest of every accepted unit (head, tree, CI run), as the SOP describes; the block PRs are then closed with links to their receipts, not merged one by one through many retargets. (4) Before merging, the owner's merge-gate checklist: the aggregate CI gate job by name on every run, a genuine stalled-versus-pending check, every comment thread re-read immediately before the command, and independent reproduction of any CI failure. (5) After merging, close the issues it fixes citing the merge commit and re-verify the tip. Never force-push `deve lop`; rollback is a revert

by merge.

4. Upstream preparation (no action taken)

State (read-only). Upstream develop is fc1e93746 and master is 6b04006b7, identical to the fork's. Upstream has open release PRs #476 (7.15), #475 (7.14.3 Bitcoin-only) and #458 (7.14.2).

Standing constraints from your notes: no upstream PR or issue for keepkey/keepkey-firmware unless you explicitly ask, always with an explicit --repo, and the fork tracker is where issues go; the release train is develop, then release branch, then master, human-gated; the fork's secret-scan history contamination (issue #544) means a PR built from fork history needs an assessment before it goes anywhere public; and the Python and device-protocol pins must be on their canonical branches with ancestry checked both ways.

Options for blocks 1 to 13: (a) update #476 to the assembled release head; (b) a new release PR replacing #476; (c) staged per-block PRs. I recommend (a) or (b) after assembly, device testing and your acceptance, with the block receipts linked from the PR body. Per-block PRs would ask upstream reviewers to review thirteen stacked PRs whose base assembly is about to change.

Go/no-go before any upstream action: assembly complete and audited; your acceptance of Blocks 11 to 13 recorded; a signed build tested on a device; Block 10's receipt reconciled; the secret-scan question settled; pins confirmed; and your explicit instruction to open or update the upstream PR.

5. Decisions needed

1. Assembly strategy (option 1 recommended) and whether to start now.
2. Whether to publish candidate/7.15.0-lineage-20260929 for device testing before assembly, on a test device only.
3. Who builds and signs a device build, and on what device and seed.
4. The upstream form, (a) or (b), decided after assembly.
5. Acceptance of Blocks 11, 12 and 13, and reconciliation of Block 10's receipt.